Legato Mobile — Flutter Frontend Study Guide

GP-Legal-AI · legato_mobile · Package: lib/ · Live: legatoappgp2026.web.app
Prepared for team discussion — block-by-block explanation of how the frontend works.

---

1. Big Picture — How the App Is Organized

The Flutter frontend talks to the FastAPI backend over HTTPS. State is managed with Provider. Navigation is simple: AuthGate decides login vs home; sub-screens use Navigator.push.

```
main.dart → Provider (AppServices, AuthProvider, ThemeNotifier)
         → AuthGate
              ■■ logged in  → HomeShell (6 bottom tabs)
              ■■ logged out → LoginScreen
         → Screens call LegatoApi → ApiClient → Backend + JWT in TokenStorage
```

Folder	Role
lib/main.dart	App entry, providers, theme, lifecycle
lib/config/	API URL, timeouts, share links
lib/storage/	Save JWT token locally
lib/api/	HTTP client + all backend endpoints
lib/services/	Auth logic (login, register, OAuth)
lib/providers/	App-wide state (user, theme)
lib/screens/	Every page the user sees
lib/widgets/	Reusable UI components
lib/theme/	LinkedIn-style light/dark theme
lib/utils/	Platform helpers (PDF download, file bytes)

Pattern used everywhere: A screen reads AppServices or AuthProvider via Provider, calls legato.someApiMethod(), then updates UI with setState or ChangeNotifier.

2. App Startup — main.dart

Block A — Initialize Flutter and config

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await RuntimeConfig.init();
  final themeNotifier = await ThemeNotifier.init();
  runApp(LegatoApp(themeNotifier: themeNotifier));
}
```

What each line does:

- ensureInitialized() — Required before async/plugin work in Flutter.
- RuntimeConfig.init() — Loads saved API URL override from device storage.
- ThemeNotifier.init() — Loads dark/light preference.
- runApp() — Starts the widget tree at LegatoApp.

Block B — Dependency injection with Provider

```
MultiProvider(
  providers: [
    Provider<AppServices>(create: (_) => AppServices()),
    ChangeNotifierProvider<AuthProvider>(... bootstrap ...),
    ChangeNotifierProvider<ThemeNotifier>.value(value: themeNotifier),
  ],
  child: MaterialApp(home: AuthGate(), ...),
)
```

Provider	Purpose
AppServices	One shared HTTP client + auth + API (JWT on every request)
AuthProvider	Current user, login/logout, session expiry
ThemeNotifier	Dark/light mode toggle

Block C — MaterialApp shell

- theme / darkTheme — LinkedIn-style gold/dark UI.
- maxWidth: 600 — On web, app looks like a centered phone column.
- home: AuthGate() — First screen decides login vs main app.

Block D — Lifecycle: when app resumes, AuthProvider.refreshUser() re-fetches /auth/me to detect expired JWT. On Android, deep links with ?token= trigger loginWithToken().

3. Configuration Layer

AppConfig — compile-time settings

```
static const String apiBaseUrl = String.fromEnvironment(
  'API_BASE_URL',
  defaultValue: 'https://srv1723974.hstgr.cloud',
);
```

Override at build: flutter build web --dart-define=API_BASE_URL=https://your-api.com
Timeouts: 90s default, 10 min analyze, 5 min chat, 2 min auth.

RuntimeConfig — runtime API URL override

Settings screen can change API URL without rebuild. Stored in SharedPreferences. Migrates old http://76.13.4.148 to HTTPS (browsers block mixed content from Firebase).

4. Storage — JWT Token (TokenStorage)

```
Key: 'access_token' in SharedPreferences
readToken()  → attach to Authorization: Bearer ...
writeToken() → after login / OAuth
clearToken() → on 401 or logout
```

5. HTTP Layer — ApiClient

Build URL + auth header

```
Uri uri(String path) => Uri.parse('${RuntimeConfig.apiBaseUrl}$path');

_headers():
  Content-Type: application/json (for POST)
  Authorization: Bearer <token>  (if logged in)
```

Handle 401 (session expired)

If statusCode == 401 → clearToken() → call onUnauthorized → AuthProvider shows 'Session expired' dialog.

Typical JSON POST pattern

postJson(path, body) → POST with JSON → parse response OR throw ApiException with server message. Screens catch ApiException and show red error text.

Also: postMultipartOcrCheck for PDF/DOCX upload to /ocr_check_and_search (long timeout).

6. AppServices — Service Wiring

```
AppServices() {
  storage = TokenStorage();
  api = ApiClient(storage: storage, onUnauthorized: ...);
  auth = AuthService(api: api, storage: storage);
  legato = LegatoApi(api);
}
```

One chain: TokenStorage → ApiClient → AuthService + LegatoApi. All screens share one token.

7. AuthService — Login / Register / OAuth

Method	Backend route	Purpose
login	POST /auth/login	Email/password → save JWT
register	POST /auth/register	Create account
verifyEmail	POST /auth/verify-email	Confirm email code
resendVerification	POST /auth/resend-verification	Resend code (with retry)
loginWithGoogleIdToken	POST /auth/google/id-token	Native Google
exchangeGoogleCode	POST /auth/google/code	Web OAuth redirect
me	GET /auth/me	Current user profile

8. AuthProvider — Auth State

bootstrap() on app start:

- Web OAuth ?code= → exchange for JWT via exchangeGoogleCode()
- Deep link ?token= → storeToken() + me()
- Saved token exists → GET /auth/me
- Sets _user or null → drives login vs home

Exposed: user, loading, isAuthenticated. Screens use context.watch<AuthProvider>() to rebuild on login/logout.

9. AuthGate — Navigation Gate

```
if (auth.loading)     → spinner
if (session expired)   → dialog "Session expired"
if (isAuthenticated)   → HomeShell
else                   → LoginScreen
```

No router package — simple if/else on home: + Navigator.push for sub-pages.

10. HomeShell — Bottom Navigation

Tab	Screen	Purpose
0 Home	DashboardTab	Quick links, stats
1 Feed	FeedScreen	Social posts
2 Network	NetworkScreen	Connections, invites
3 Contracts	ContractsTabScreen	User analyses list
4 Alerts	AlertsScreen	Notifications
5 Profile	ProfileScreen	Own profile edit

IndexedStack keeps all tabs in memory — switching tabs is instant; scroll position is preserved.

11. LegatoApi — API Facade

Group	Example methods	Backend
Analysis	analyzeContract, listAnalyses	/ocr_check_and_search, /analyses
Chat	chatMessage, chatAssistant	/chat/*
Social	getFeed, createPost, sendNetworkInvite	/api/posts, /api/network/*
Profile	getSocialProfile, putApiProfile	/api/profile/*
Admin	adminListUsers, adminDeleteUser	/analyses/admin/*

Resilient helpers: getSocialProfileResilient tries /api/profile/{id}, falls back to /legato/profile/me shape if 404.

12. Standard Screen Pattern

```
class _SomeScreenState extends State<SomeScreen> {
  bool _loading = true;
  String? _err;
  List<dynamic> _data = [];

  void initState() { super.initState(); _load(); }

  Future<void> _load() async {
    setState(() { _loading = true; _err = null; });
    try {
      final data = await context.read<AppServices>().legato.someMethod();
      setState(() { _data = data; _loading = false; });
    } on ApiException catch (e) {
      setState(() { _err = e.message; _loading = false; });
    }
  }

  Widget build() {
    if (_loading) return CircularProgressIndicator();
    if (_err != null) return Text(_err!);
    return ListView(...);
  }
}
```

13. Feature Screens Summary

Auth (lib/screens/auth/)

login_screen — email/password; Google OAuth (web redirect vs mobile browser).
register_screen — sign up → verify email.
verify_email_screen — enter code; resend code.
forgot_password_screen — reset password flow.
Google on web: OAuth → redirect to Firebase with ?code= → bootstrap exchanges for JWT.

Analyze (analyze_screen.dart)

FilePicker → analyzeContract(bytes) → backend OCR + RAG + ML + LLM → optional save → AnalysisDetailScreen. WakelockPlus prevents sleep during long analysis.

Social (lib/screens/social/)

feed_screen — posts, like/comment/share (WhatsApp uses AppConfig.shareBaseUrl).
network_screen — suggestions, invites, connections.
member_profile_screen — view user; Connect/Connected via connection_status.
profile_screen — edit own profile, avatar, skills.

Chat, Admin, More

chat_assistant_screen — POST /chat/assistant.
chat_analysis_screen — chat about saved analysis.
admin_screen — list users, change roles, delete users (admin only).
more_screen — links to Features hub, Settings, Roadmap, Admin.

14. Reusable Widgets

Widget	File	Role
LegatoAppBar	legato_app_bar.dart	Top bar with back button
UserAvatar	user_avatar.dart	Circle avatar from URL or initials
AnalysisIdPicker	analysis_id_picker.dart	Dropdown of user analyses
DocumentChatBubble	document_chat_bubble.dart	Chat message UI

15. Theme (linkedin_theme.dart)

Gold accent (navActiveGold), dark scaffold colors, card styles. textSecondaryAdaptive(context) works in light and dark mode.

16. Platform Utils (lib/utils/)

Conditional imports for web vs mobile:
text_download.dart → web or io implementation
pdf_download.dart — same pattern
platform_file_bytes.dart — gallery/camera bytes on web vs native

17. End-to-End: Admin Deletes User

```
AdminScreen._deleteUser()
  → showDialog confirm
  → legato.adminDeleteUser(userId)
    → ApiClient.deleteJson('/analyses/admin/users/$id')
  → _load() refreshes list
  → SnackBar "Deleted email@..."
```

18. End-to-End: View Friend Profile

```
MemberProfileScreen._load()
  → getSocialProfileResilient(userId)
  → if no connection_status, check getNetworkConnections()
  → UI shows "Connected" instead of "Connect"
```

19. How to Read Any Screen Quickly

1. Find initState / _load() — what API is called?
2. Find context.read() — which legato.* method?
3. Find build() — loading / error / success UI
4. Find Navigator.push — where can user go next?

20. Discussion Talking Points

• Why Provider instead of Riverpod/Bloc? — Simple shared services + auth state; matches small team scope.

• Why IndexedStack for tabs? — Preserve state; faster tab switching.

• How is JWT security handled? — Stored in SharedPreferences; cleared on 401; not in URL except OAuth handoff.

• Web vs mobile differences? — Google OAuth redirect on web; deep links on Android; maxWidth 600 on web.

• How does the app survive backend changes? — Resilient API helpers (getSocialProfileResilient).

• Where is the API URL configured? — AppConfig compile-time + RuntimeConfig runtime override in Settings.

• What happens when analysis takes 5+ minutes? — longTimeout (10 min), WakelockPlus, progress status text.

Generated from GP-Legal-AI Flutter frontend (legato_mobile). Repo: lib/ · 61 Dart files · Live: legatoappgp2026.web.app